

CricTactix-AI: Backend Engineering from Scratch

A Senior Engineer's Step-by-Step Design, Code, & Reasoning Guide

Welcome! If you were to design and build a state-of-the-art backend like **CricTactix-AI** from absolute scratch, this guide is your interactive masterclass.

We will not just look at code; we will analyze **why** certain architectures are selected, solve **logical system design questions**, and walk through building every single layer of a modern AI-powered data platform step by step.

■ Roadmap of the Learning Journey

1. **The Blueprint:** Defining the tech stack and system architecture.
 2. **Step 1: Relational Schema & Normalization** (Designing the database).
 3. **Step 2: Declarative ORM & Infrastructure** (SQLAlchemy 2.0 & SQLite setup).
 4. **Step 3: High-Performance Ingestion (ETL)** (Bulk-parsing matches in seconds).
 5. **Step 4: FastAPI REST API Services** (Paginated search & live scorecards).
 6. **Step 5: The LangChain AI SQL Agent** (Natural language to SQL compilation).
 7. **Step 6: Isolation & Verification** (Automated Pytest suite).
-

■■ The Blueprint: Architecture Philosophy

Before writing a single line of code, we must choose our tools and define how the components communicate.

Logical Question 1

> "Why did we choose Python, FastAPI, SQLite, and SQLAlchemy over Node.js with MongoDB for a cricket analytics app?"

>

> **The Senior Engineer's Answer:**

> - **MongoDB (Document Database) vs. SQLite/PostgreSQL (Relational):** Cricket data is deeply relational. A single ball (**CODE_PLACEHOLDER_0**) belongs to a **CODE_PLACEHOLDER_1**, is bowled by a **CODE_PLACEHOLDER_2** (who belongs to a **CODE_PLACEHOLDER_3**), faced by another **CODE_PLACEHOLDER_4**, and occurs in a specific **CODE_PLACEHOLDER_5**. In MongoDB, representing these relationships requires either massive, redundant duplication (de-normalization) or slow client-side joins. A relational schema with foreign key constraints ensures **strict consistency** and allows the database engine to run high-performance relational queries (e.g., player strike rates).

> - **Python + FastAPI vs. Node.js:** Python is the undisputed king of Data Science and AI. Using Python allows us to run standard ETL libraries and natively integrate with LLM orchestration frameworks like **LangChain** and **OpenAI**. **FastAPI** is selected because it is built on asynchronous ASGI (**CODE_PLACEHOLDER_0**), meaning it matches Node.js's concurrent performance while providing automatic OpenAPI documentation and strict data type validation via **Pydantic**.

■ Step 1: Database Normalization & Schema Design

Our goal is to represent cricket matches at a ball-by-ball micro-level. A simple flat spreadsheet would quickly become unmanageable. We need to normalize our data.

Logical Question 2

> "What is the danger of storing player information directly inside the **CODE_PLACEHOLDER_0** table as a plain string, e.g. **CODE_PLACEHOLDER_1**?"

>

> **The Senior Engineer's Answer:**

> Doing so violates **Third Normal Form (3NF)** and leads to **data anomalies**:

> 1. **Update Anomaly:** If Babar Azam changes his spelling or profile, we would have to search and update millions of rows in the **CODE_PLACEHOLDER_0** table.

> 2. **Insertion Anomaly:** We cannot store a player's profile (e.g., date of birth or batting hand) until they have played at least one delivery.

> 3. **Spelling Variations:** **CODE_PLACEHOLDER_0**, **CODE_PLACEHOLDER_1**, and **CODE_PLACEHOLDER_2** would be treated as three different players, breaking statistical accuracy.

>

> **The Solution:** Create a dedicated **CODE_PLACEHOLDER_0** table with a unique integer ID, and map matches/deliveries to it using relationships.

The 6-Table Relational Schema

We will design a normalized schema containing six tables:

1. **CODE_PLACEHOLDER_0**: Canonical record of teams (e.g. Lahore Qalandars).
 2. **CODE_PLACEHOLDER_0**: Canonical profile of players.
 3. **CODE_PLACEHOLDER_0**: Bridges a team to a specific calendar year.
 4. **CODE_PLACEHOLDER_0**: Roster mapping of players to a specific team's season.
 5. **CODE_PLACEHOLDER_0**: Metadata about match events (venue, dates, teams playing).
 6. **CODE_PLACEHOLDER_0**: Highly granular ball-by-ball actions (~250 per match).
-

■■ Step 2: Declarative ORM & Infrastructure Setup

Now let's build the directory structure and initialize our database connection.

Directory Structure

Create this structure to separate your application's concerns strictly:

```
Backend/
  ■■■ main.py                # Entry point
  ■■■ requirements.txt       # Package dependencies
  ■■■ app/
    ■ ■■■ core/              # Config and database engines
    ■ ■ ■ ■ config.py
    ■ ■ ■ ■ database.py
    ■ ■■■ models/           # Database ORM models
    ■ ■ ■ ■ base.py
    ■ ■ ■ ■ team.py
    ■ ■ ■ ■ player.py
    ■ ■ ■ ■ match.py
    ■ ■■■ schemas/          # Request/Response validation contracts
    ■ ■■■ data_pipeline/    # ETL data parser
```

Initializing the Database (**CODE_PLACEHOLDER_0**)

We will use **SQLAlchemy 2.0** with Type Annotation support.

create_engine: Establishes connection pool

check_same_thread=False is required ONLY for SQLite in multi-threaded contexts

SessionLocal: Factory class to generate new database sessions

Base: Parent class that all ORM models will inherit from

Dependency generator to inject database sessions into our web routes

```
from sqlalchemy import create_engine
from sqlalchemy.orm import declarative_base, sessionmaker
DATABASE_URL = "sqlite:///./cricket_ai.db"
engine = create_engine(
    DATABASE_URL, connect_args={"check_same_thread": False}
)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

Defining Relational Models (*CODE_PLACEHOLDER_0*)

Here is how we define the **CODE_PLACEHOLDER_0** model using SQLAlchemy 2.0's **CODE_PLACEHOLDER_1** syntax:

```
from sqlalchemy import Integer, String, JSON
from sqlalchemy.orm import Mapped, mapped_column
from app.core.database import Base
class Team(Base):
    __tablename__ = "teams"
    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    name: Mapped[str] = mapped_column(String, unique=True, index=True)
    short_name: Mapped[str] = mapped_column(String, nullable=True)
    aliases: Mapped[dict] = mapped_column(JSON, nullable=True)
```

■ Step 3: High-Performance Ingestion Pipeline (ETL)

Now, suppose we have hundreds of matches stored as local raw JSON files (from Cricsheet). Each file contains match details and a nested list of deliveries. If we loop and insert each ball using simple SQL statements, it will take ages.

Logical Question 3

> "Why does calling **CODE_PLACEHOLDER_0** followed by **CODE_PLACEHOLDER_1** inside a loop of 25,000 deliveries make our application extremely slow, and how do we resolve this?"

>

> **The Senior Engineer's Answer:**

> Calling **CODE_PLACEHOLDER_0** forces the database engine to write the transaction directly to the hard disk (Physical I/O). Hard disks are thousands of times slower than system memory.

> To achieve high performance, we use three key techniques:

> 1. **In-Memory Caching:** Pre-fetch existing Teams and Players from the database *once* into memory (e.g. standard Python dictionaries). Use these dictionaries to look up IDs instantly without querying the database in the loop.

> 2. **CODE_PLACEHOLDER_0 instead of CODE_PLACEHOLDER_1:** Flush writes the data to the database's memory buffers and generates primary key IDs (so we can link **CODE_PLACEHOLDER_2** to deliveries) without performing expensive disk writes.

> 3. **CODE_PLACEHOLDER_0:** Instead of individual inserts, we accumulate all deliveries in a list and insert them in large chunks.

> 4. **Single Atomic Commit:** We call **CODE_PLACEHOLDER_0** only *once* at the very end of our migration script.

Implementing the ETL Core

Here is the core logic of our parser (**CODE_PLACEHOLDER_0**):

```
import os
import json
from app.core.database import SessionLocal
from app.models.team import Team
from app.models.player import Player
def load_match_data(file_path: str):
    db = SessionLocal()
    try:
        with open(file_path, "r") as f:
            data = json.load(f)
        # 1. Pre-fetch and cache existing teams
        existing_teams = {t.name: t for t in db.query(Team).all()}
        # 2. Extract and resolve teams from JSON
        team_names = data["info"]["teams"]
        resolved_team_ids = []
        for name in team_names:
            if name not in existing_teams:
                new_team = Team(name=name)
                db.add(new_team)
                db.flush() # Flushes to memory, generates new_team.id
                existing_teams[name] = new_team
            resolved_team_ids.append(existing_teams[name].id)
        # 3. Repeat caching process for players, seasons, and matches...
        db.commit() # Atomic write to disk
        print("■ Match parsed and saved successfully!")
    except Exception as e:
        db.rollback() # Safely roll back if any error occurs
        raise e
    finally:
        db.close()
```

■ Step 4: The Web REST API & Scorecard Generator

Now that our database is seeded, we want to build REST endpoints using **FastAPI** so our frontend can fetch match lists and detailed scorecards.

Logical Question 4

> "Why is it better to calculate scorecard statistics (runs, strike rates, wickets) dynamically from raw delivery data when requested, rather than pre-calculating and saving them directly into a `CODE_PLACEHOLDER_0` table?"

>

> **The Senior Engineer's Answer:**

> Calculating statistics dynamically keeps the schema flexible. If the logic for calculating strike rate changes, or if we decide to track a new metric (like "runs scored in the death overs"), we don't have to migrate our database tables. We simply update the aggregation query code.

>

> *Note: If our database grows to millions of matches and API latency becomes an issue, we can implement a read-model caching layer using Redis or store pre-calculated scorecard JSON documents on the `CODE_PLACEHOLDER_0` table directly upon ingestion.*

Implementing the FastAPI Router (`CODE_PLACEHOLDER_0`)

Here is how we set up the FastAPI router and implement a paginated search endpoint:

```
from fastapi import APIRouter, Depends, Query
from sqlalchemy.orm import Session
from typing import List
from app.core.database import get_db
from app.models.match import Match
router = APIRouter(prefix="/api/matches", tags=["Matches"])
@router.get("/")
def get_matches(
    team: str = Query(None, description="Filter matches by team name"),
    page: int = Query(1, ge=1),
    limit: int = Query(20, le=100),
    db: Session = Depends(get_db)
):
    query = db.query(Match)
    if team:
        # Asymmetrical search: check if team is team_1 OR team_2
        query = query.filter(
            (Match.team_1.like(f"%{team}%")) |
            (Match.team_2.like(f"%{team}%"))
        )
    # Implement pagination
    offset = (page - 1) * limit
    matches = query.order_by(Match.start_date.desc()).offset(offset).limit(limit).all()
    return {
```

```
"page": page,
"limit": limit,
"results": matches
}
```

■ Step 5: The LangChain AI SQL Agent

Now comes the absolute superpower of CricTactix-AI: allowing users to ask questions in plain English (e.g. *"Who had the highest strike rate in the 2024 Powerplay?"*) and converting it to SQL.

Logical Question 5

> "If an AI agent can write any SQL statement and execute it on our database, how do we prevent a malicious user from typing: 'Show me my profile and then drop table users'?"

>

> **The Senior Engineer's Answer:**

> We enforce multiple security boundaries:

> 1. **Read-Only Database Roles:** In production, the credentials we provide to the AI agent must belong to a read-only user account. This user is strictly limited to running **CODE_PLACEHOLDER_0** statements and physically cannot run **CODE_PLACEHOLDER_1**, **CODE_PLACEHOLDER_2**, or **CODE_PLACEHOLDER_3** commands.

> 2. **Instruction Guardrails:** We provide a rigid System Prompt to the LLM agent, forcing it to reject any statements that attempt modifying operations.

> 3. **Input Parameterization:** Under the hood, LangChain translates the queries via SQLAlchemy, which compiles parameters securely, preventing traditional string-concatenation SQL Injection.

Building the SQL Agent (**CODE_PLACEHOLDER_0**)

Here is how we build the SQL Agent using **LangChain** and **OpenAI**:

Initialize LangChain SQL wrapper for our database

Initialize ChatOpenAI with temperature 0 for extreme determinism

Custom instructions to guide the LLM on business logic

Create the SQL Agent

```
from fastapi import APIRouter, Depends
from langchain_community.utilities import SQLDatabase
from langchain_community.agent_toolkits import create_sql_agent
from langchain_openai import ChatOpenAI
from app.core.config import settings
router = APIRouter(prefix="/api/chat", tags=["AI Agent"])
db_engine = SQLDatabase.from_uri("sqlite:///cricket_ai.db")
llm = ChatOpenAI(
    model="gpt-4o-mini",
    temperature=0,
    api_key=settings.OPENAI_API_KEY
)
AGENT_PREFIX = """
You are a senior cricket analytics agent. You have access to a database containing 'teams', 'players', 'matches'
When asked for statistics:
1. Always use case-insensitive SQL LIKE queries for player and team names.
2. Batting strike rates are calculated as: (sum(runs_batter) * 100.0) / count(deliveries where runs_wides is 0)
3. If you cannot answer using the tables provided, say so clearly. Do not make up information.
"""
agent_executor = create_sql_agent(
    llm=llm,
    db=db_engine,
    prompt=AGENT_PREFIX,
    verbose=True,
    handle_parsing_errors=True
)
@router.post("/")
def ask_analytics_agent(question: str):
    try:
        response = agent_executor.run(question)
        return {"status": "success", "answer": response}
    except Exception as e:
        return {"status": "error", "message": str(e)}
```

■ Step 6: Isolation & Verification (Testing)

A backend is only as reliable as its test suite. We must write automated tests that run in complete isolation.

Logical Question 6

> "Why must we use a different database file (e.g. `CODE_PLACEHOLDER_0`) when running tests, instead of our active `CODE_PLACEHOLDER_1` database?"

>

> **The Senior Engineer's Answer:**

> Tests perform writes, updates, and deletes to verify functions behave correctly. If tests ran on our main database, they would corrupt active data, leaving behind test players or deleting actual matches. Using a separate, isolated test database built from scratch and torn down after each test run ensures **repeatable, isolated, and safe test execution**.

Writing a Pytest Fixture (`CODE_PLACEHOLDER_0`)

We set up a fixture to create tables in our isolated test database before each test runs:

```
import pytest
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker
from app.core.database import Base
from app.core.database import get_db
TEST_DATABASE_URL = "sqlite:///./test_cricket_ai.db"
engine = create_engine(TEST_DATABASE_URL, connect_args={"check_same_thread": False})
TestingSessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
@pytest.fixture(scope="function")
def db_session():
    # 1. Build the fresh schema
    Base.metadata.create_all(bind=engine)
    db = TestingSessionLocal()
    try:
        yield db
    finally:
        db.close()
    # 2. Tear down everything
    Base.metadata.drop_all(bind=engine)
```

■ Next Steps in Your Learning Journey

Now that you understand the architectural pipeline, you can progress step by step:

1. **Explore the codebase:** Check out **`CODE_PLACEHOLDER_0`** and notice how tables are structured.
2. **Review the ETL logic:** Look at **`CODE_PLACEHOLDER_0`** to see the actual Python loops.
3. **Experiment with the AI:** Try querying endpoints locally or sending chat queries to see the LangChain executor print the raw generated SQL to your console.

This structure forms the backbone of highly scalable, modern, data-driven platforms. Happy coding!